MINOR PROJECT REPORT

E-COMMERCE CUSTOMER SEGMENTATION USING MACHINE LEARNING

Submitted in partial fulfilment of the requirements for the degree of

B.Tech – Computer Science & Engineering (Artificial Intelligence & Machine Learning)

Submitted by

PARISHTI

Rungta College of Engineering & Technology

Chhattisgarh Swami Vivekanand Technical University (CSVTU)

Academic Session: 2026–2027

CERTIFICATE

This is to certify that the minor project report entitled “E-Commerce Customer Segmentation Using Machine Learning” is submitted by PARISHTI, student of B.Tech – Computer Science & Engineering (Artificial Intelligence & Machine Learning), Rungta College of Engineering & Technology, in partial fulfilment of the academic requirements. The work is presented for academic evaluation.

Project Guide: _____

Head of Department: _____

Date: _____ Place: _____

DECLARATION

I, PARISHTI, declare that this report titled “E-Commerce Customer Segmentation Using Machine Learning” has been prepared for academic purposes based on the implementation available in my project repository. Sources and tools used in preparing the report are acknowledged in the references. Results quoted from the repository are identified as sample outputs and should be re-generated before final evaluation.

Signature: _____

Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my institution and faculty members for providing the opportunity to undertake this minor project. I am thankful to my project guide for guidance and feedback. I also acknowledge the developers and maintainers of Python and the open-source libraries used in this project, including pandas, NumPy, Matplotlib, Seaborn and scikit-learn. Finally, I thank my family and friends for their encouragement during the completion of this work.

ABSTRACT

Customer segmentation helps e-commerce businesses understand differences in purchasing behaviour and plan more relevant customer engagement. This project develops a Python-based workflow using RFM analysis with K-Means clustering. Transaction data is validated and cleaned, total transaction value is calculated, and customer-level RFM features are created. Log transformation and standardisation prepare features for clustering. The program explores inertia for candidate cluster counts, fits a four-cluster K-Means model, calculates a silhouette score and assigns business-readable labels using heuristic rules. Visualisations and CSV files summarise the results. The repository's committed sample summary represents 20 customers across four segments: 9 At-Risk, 6 Loyal, 2 Potential and 3 VIP customers. These are demonstration results from a small sample, not evidence of performance on a large commercial dataset.

Keywords: E-commerce, Customer Segmentation, Machine Learning, RFM Analysis, K-Means, Python, Unsupervised Learning.

TABLE OF CONTENTS

Certificate

Declaration

Acknowledgement

Abstract

Chapter 1: Introduction

Chapter 2: Literature Review and Theoretical Background

Chapter 3: Requirement Analysis

Chapter 4: System Design and Methodology

Chapter 5: Implementation

Chapter 6: Results and Discussion

Chapter 7: Testing and Validation

Chapter 8: Conclusion and Future Scope

References

Appendix A: Repository Files

Appendix B: Important Code Logic

Appendix C: Viva-Voce Questions

Note: Page numbers may shift after editing. Use the document headings as the navigation structure.

LIST OF FIGURES AND TABLES

Figure 1.1: Overall project workflow

Figure 4.1: Data processing and modelling pipeline

Figure 6.1: Customer count by segment

Figure 6.2: Average monetary value by segment

Figure 6.3: Average recency by segment

Table 4.1: Input data fields

Table 6.1: Customer segment summary

Table 7.1: Suggested test cases

CHAPTER 1: INTRODUCTION

1.1 Background

Electronic commerce allows customers to browse products, place orders and make payments through digital platforms. Each transaction creates useful information such as product, quantity, price, date, invoice number and customer identifier. Analysing this information helps a business understand how customers purchase and how their behaviour changes over time.

Customer segmentation is the process of grouping customers who show similar behaviour or characteristics. Instead of treating every customer in the same way, a business can develop different strategies for frequent purchasers, high-value customers, recently active customers and customers who may have stopped buying. In this project, segmentation is performed using transaction behaviour rather than demographic attributes.

1.2 Project Overview

The project uses Python to load retail transaction data, clean invalid records, calculate customer-level Recency, Frequency and Monetary (RFM) features, standardise the features and apply the K-Means clustering algorithm. The program then assigns understandable business labels such as VIP Customers, Loyal Customers, Potential Customers and At-Risk Customers. It also creates visualisations and saves customer-level and segment-level CSV reports.

The GitHub repository includes the main Python script, a small retail dataset in CSV and Excel formats, a dataset-generation script, a requirements file and generated segment output files. The committed sample output represents 20 customers and four named segments. These results should be interpreted as a demonstration on a small sample, not as a production-scale retail study.

1.3 Problem Statement

Retailers may hold large amounts of transaction data but lack a simple, repeatable process to turn the data into customer groups. Manual analysis can be slow and inconsistent. The problem addressed here is to develop a small, reproducible machine-learning workflow that transforms transaction records into customer-level behavioural features, discovers groups with similar RFM patterns and presents the results in a form that can guide marketing decisions.

1.4 Objectives

The main objectives are: (1) load retail transactions from CSV or Excel; (2) validate required fields and inspect data quality; (3) remove duplicates, missing customer information, cancellations and invalid quantity or price values; (4) calculate transaction value; (5) aggregate transactions into RFM features; (6) use log transformation and standardisation to prepare features; (7) cluster customers using K-Means; (8) evaluate separation using the silhouette score; (9) create charts and summaries; and (10) suggest marketing actions for the resulting segments.

1.5 Scope

The current scope is an offline Python analysis of a small retail transaction dataset. It demonstrates customer grouping and report generation. It does not include a live shopping website, customer login, payment gateway, real-time database, automated email delivery or a deployed prediction API. The program is designed as an educational AIML minor project and can be extended with larger and more representative data.

1.6 Significance

The project connects core data-science concepts with a business use case. It demonstrates data cleaning, feature engineering, exploratory analysis, unsupervised learning, evaluation, visualisation and interpretation. The output can help explain how different customer groups might be approached differently, while also showing the limitations of small datasets and automatically assigned labels.

CHAPTER 2: LITERATURE REVIEW AND THEORETICAL BACKGROUND

2.1 Customer Segmentation

Customer segmentation is widely used in marketing analytics to divide a customer base into groups that are internally similar and meaningfully different from other groups. Segments may be based on geography, demographics, preferences or observed behaviour. Transaction-based segmentation is useful when reliable purchase histories are available.

2.2 RFM Analysis

RFM is a behavioural framework built from three measures. Recency indicates how long it has been since a customer last purchased. Frequency measures how often the customer purchased during the observation period. Monetary measures the total spending attributed to the customer. RFM is popular because it is relatively easy to explain and can be calculated from invoice-level data.

A low recency value generally means the customer purchased recently. A high frequency value means the customer made more distinct purchases. A high monetary value means the customer generated more sales value. These measures should be interpreted in the context of the data collection period, product mix, currency, returns policy and business model.

2.3 Unsupervised Learning

Unsupervised learning discovers structure in data without using pre-labelled target classes. Clustering is one form of unsupervised learning. It groups observations according to a distance or similarity measure. In this project, the algorithm receives the RFM features and discovers groups; the labels VIP, Loyal, Potential and At-Risk are assigned afterwards using business-oriented rules.

2.4 K-Means Clustering

K-Means partitions observations into K clusters. It starts with cluster-centre estimates, assigns each observation to the nearest centre, updates each centre to the mean of its assigned observations, and repeats until the assignments or centres stabilise. The objective is to minimise the within-cluster sum of squared distances.

K-Means is computationally efficient and easy to implement, but it has limitations. The number of clusters must be chosen, results can depend on initialisation, and distance-based clustering can be affected by scale and outliers. It tends to work best when clusters are reasonably compact in the chosen feature space.

2.5 Feature Scaling and Log Transformation

RFM features have different units and distributions. Monetary values may be much larger numerically than frequency values, so standardisation is used to put features on a comparable scale. The project applies log1p, which calculates log(1+x), to reduce the influence of large positive values and then applies StandardScaler. The scaled features are used for clustering while the original RFM values are retained for business summaries.

2.6 Cluster Evaluation

The code computes inertia for candidate values of K from 2 through 10 and plots the elbow curve. Inertia measures the within-cluster squared distance; it normally decreases as K increases. The elbow method looks for a point after which additional clusters provide smaller improvements. The current script then uses K=4. It also calculates a silhouette score, which compares how close each point is to its own cluster versus neighbouring clusters. A score near 1 suggests well-separated clusters, a score near 0 suggests overlap, and a negative score may indicate poor assignment. The exact score should be taken from a fresh run of the code, not assumed from the committed summary CSV.

2.7 Summary of the Approach

The project combines an interpretable RFM framework with a general-purpose clustering algorithm. RFM supplies business-readable features; K-Means supplies data-driven group assignments; descriptive statistics and visualisations support interpretation. The approach is appropriate for a learning project, but real deployment would require validation with more data and domain experts.

CHAPTER 3: REQUIREMENT ANALYSIS

3.1 Functional Requirements

The system shall read the available CSV dataset, or use the Excel file if the CSV file is absent. It shall validate the required columns, display basic data-quality information, clean unusable rows, calculate TotalAmount, generate RFM features, run K-Means clustering, assign business segment labels, create visualisations, print key customer lists and save CSV output files.

3.2 Non-Functional Requirements

The system should be understandable, reproducible, reasonably efficient for small datasets and easy to run on a student computer. It should give clear errors when a dependency or dataset is missing. Results should be reproducible where possible through a fixed random_state. The script should not expose private customer information in public repositories; any real customer data should be anonymised and handled according to institutional and legal requirements.

3.3 Hardware Requirements

A standard laptop or desktop capable of running Python is sufficient for the small sample dataset. A multi-core processor and several gigabytes of RAM are adequate for this demonstration. Larger datasets may require more memory and performance testing. No specialised GPU is required because the project uses classical machine learning rather than deep neural networks.

3.4 Software Requirements

Python 3 with pandas, NumPy, Matplotlib, Seaborn, scikit-learn and openpyxl is required. Visual Studio Code may be used as the development environment. Git and GitHub are used for version control and project sharing. The repository's requirements file pins package versions; if a package version is incompatible with the installed Python version, a compatible Python environment and package set should be selected.

3.5 Feasibility Study

Technical feasibility is high because the workflow uses common Python libraries and runs locally. Economic feasibility is high because the tools are open source and no paid service is required. Operational feasibility is suitable for a student project because the inputs and outputs are simple files. Organisational feasibility for a real company would require approval for data access, privacy review, marketing validation and ongoing monitoring.

3.6 Use Cases

A student or analyst can run the script, inspect the printed dataset statistics, view the plots, review the segment summary and open the exported CSV files. A marketing analyst can use the segment names as a starting point for campaign planning. A project evaluator can review the code, dependencies, methodology and output files in the repository.

CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

Figure 4.1: Processing pipeline

Transaction CSV / Excel → Validation and Cleaning → RFM Feature Engineering → Log Transformation and Scaling → K-Means Clustering → Business Segment Labels → Charts and CSV Reports

4.1 System Architecture

The pipeline has six logical stages: input and validation; preprocessing; RFM feature engineering; scaling and clustering; interpretation and segment assignment; and output generation. Data flows from the transaction file into a cleaned DataFrame, then into one row per customer. The clustering stage adds a numeric cluster identifier, and the interpretation stage maps each cluster to a business label. Finally, charts and CSV files are produced.

4.2 Input Data Schema

The program expects the following fields: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID and Country. InvoiceNo identifies an invoice; StockCode identifies a product; Description gives the product name; Quantity indicates units purchased; InvoiceDate records the transaction time; UnitPrice records price per unit; CustomerID identifies a customer; and Country provides a country label. The script checks for the required column names before proceeding.

Field	Meaning	Use
InvoiceNo	Invoice identifier	Count invoices; identify cancellations
StockCode	Product identifier	Product-level reference
Description	Product name	Readable product description
Quantity	Units in transaction	Transaction value
InvoiceDate	Date and time	Recency and trends
UnitPrice	Price per unit	Transaction value
CustomerID	Customer identifier	Customer aggregation
Country	Country label	Geographical summary

4.3 Data Cleaning

Duplicate rows are removed. Records missing CustomerID, InvoiceDate, Quantity or UnitPrice are excluded. CustomerID is converted to numeric and then to an integer. Invoice numbers beginning with C are treated as cancellation records and excluded. Quantity and UnitPrice are converted to numeric values; rows with non-positive quantity or price are removed. InvoiceDate is parsed as a date, and rows with invalid dates are excluded. The transaction value is calculated as Quantity multiplied by UnitPrice.

These choices are practical for the sample project, but business-specific data may require more nuanced treatment. For example, negative quantities may represent returns that should be accounted for rather than simply removed. A production system should document such

decisions and test their impact.

4.4 RFM Feature Engineering

The analysis date is defined as one day after the latest valid invoice date. Recency is the number of days between this analysis date and the customer's most recent invoice. Frequency is the number of unique invoices for the customer. Monetary is the sum of TotalAmount for that customer. These three measures are concatenated into a customer-level table with CustomerID as the index.

4.5 Scaling and Clustering

The project copies the RFM table, applies log1p to Recency, Frequency and Monetary, and standardises each feature using StandardScaler. The script calculates inertia for K values from 2 to 10 and plots an elbow curve. It then sets optimal_k to 4 and fits KMeans with random_state=42 and n_init=10. The cluster assignment is added to the original RFM table. The current value K=4 is a project design choice; for rigorous analysis it should be confirmed with elbow, silhouette, stability and business interpretability checks.

4.6 Segment Naming Logic

The code computes a BusinessScore using the ranks of average Frequency and Monetary, minus the rank of average Recency. The cluster with the highest score is labelled VIP Customers. The cluster with the highest average Recency is labelled At-Risk Customers. Among the remaining clusters, the one with the highest average Frequency is labelled Loyal Customers. Any other remaining cluster is labelled Potential Customers. These are heuristic labels, not supervised model predictions. They should be checked against cluster statistics and refined when clusters overlap or the dataset changes.

4.7 Output Design

The script saves customer-level results to customer_segments.csv and aggregated segment statistics to customer_segment_summary.csv. It also prints total customers, number of orders, total revenue, cluster count, silhouette score, segment counts, top-spending customers, most-frequent customers, at-risk customers and marketing recommendations. Several charts are generated to communicate the findings.

4.8 Algorithm

Algorithm: (1) Load transaction file. (2) Validate required columns. (3) Remove duplicates and invalid rows. (4) Calculate TotalAmount. (5) Aggregate Recency, Frequency and Monetary by CustomerID. (6) Apply log1p and standard scaling. (7) Examine inertia for K=2 to 10. (8) Fit K-Means with K=4. (9) Add cluster labels. (10) Calculate cluster means and silhouette score. (11) Map clusters to business segment names. (12) Plot and export summaries.

4.9 Pseudocode

START
Read retail data
Validate required columns
Remove duplicates and invalid transactions
Calculate TotalAmount = Quantity × UnitPrice

For each customer, calculate Recency, Frequency, Monetary
Log-transform RFM features and standardise them
For K from 2 to 10, fit K-Means and record inertia
Fit final K-Means with K = 4
Calculate cluster summaries and silhouette score
Assign business segment names using RFM ranks
Generate charts and customer highlights
Save customer-level and segment-level CSV files
END

CHAPTER 5: IMPLEMENTATION

5.1 Repository Structure

The public GitHub repository contains the main script, a dataset-generation helper, the retail CSV and Excel files, the requirements file and two result CSV files. The main file is named “E-COMMERCE CUSTOMER SEGMENTATION.py”. The README currently contains only a short project heading, so it should be expanded before final submission with setup steps, project objectives, output descriptions and screenshots.

5.2 Main Modules

The implementation is organised into functions. load_dataset reads the input and prints basic information. validate_columns checks the schema. preprocess_data cleans records and computes transaction values. build_rfm creates customer-level features. Plotting functions show country distribution, monthly sales and RFM distributions. build_clusters scales features, selects the fixed cluster count, fits K-Means and evaluates it. assign_segments maps clusters to business names. Other functions display segment summaries, customer highlights and recommendations. main coordinates the complete pipeline.

5.3 Libraries Used

pandas is used for tabular data processing and grouping. NumPy provides numerical transformations such as log1p. Matplotlib creates plots. Seaborn provides statistical charts. scikit-learn provides StandardScaler, KMeans and silhouette_score. openpyxl allows pandas to read Excel workbooks. pathlib provides paths relative to the script so the project does not depend on a hard-coded personal directory.

5.4 Running the Project

Recommended steps: install a compatible Python 3 version; open the repository folder in VS Code; create and activate a virtual environment; install dependencies from requirements.txt; confirm that Online Retail.csv or Online Retail.xlsx is in the project folder; then run the main Python file. On Windows, a typical setup is `py -m venv .venv`, `.venv\Scripts\activate`, `python -m pip install -r requirements.txt`, and `python "E-COMMERCE CUSTOMER SEGMENTATION.py"`. Use the interpreter from the same virtual environment in VS Code. Exact package compatibility should be verified for the chosen Python version.

5.5 Error Handling

The script raises a clear ImportError when required libraries are missing and reports the interpreter path. It raises FileNotFoundError if neither dataset file is present, and ValueError if required columns are missing. This is useful for beginner troubleshooting. More robust production handling could add structured logging, validation reports, exception handling around file parsing and automated tests.

5.6 Reproducibility

The clustering code sets random_state=42 and n_init=10, which helps make results repeatable under the same data and software environment. However, library versions, data changes and preprocessing changes can still affect the output. A submitted report should record the run date,

Python version, dependency versions, dataset provenance and exact generated results.

CHAPTER 6: RESULTS AND DISCUSSION

6.1 Dataset and Output Caveat

The committed output files in the repository show a demonstration with 20 customer records and four segments. The summary CSV is the basis for the descriptive tables in this report. The repository also contains a small dataset-generation script. The dataset is too small to support broad claims about actual e-commerce customer behaviour. Results should be refreshed by running the current code in the intended environment before a final academic submission.

6.2 Segment Summary

The committed customer_segment_summary.csv reports 9 At-Risk Customers, 6 Loyal Customers, 2 Potential Customers and 3 VIP Customers. These counts total 20 customers. The segment percentages are 45%, 30%, 10% and 15%, respectively. Since this is a small demonstration dataset, a change in only a few customers can noticeably alter the percentages.

Segment	Customers	Avg. Recency	Avg. Frequency	Avg. Monetary
At-Risk	9	14.22	1.00	70.76
Loyal	6	12.00	2.17	138.90
Potential	2	2.50	1.00	37.80
VIP	3	3.33	2.00	203.00

6.3 Average RFM Values

The output reports the following averages. At-Risk Customers: average recency 14.22 days, frequency 1.00 invoices and monetary value 70.76. Loyal Customers: average recency 12.00 days, frequency 2.17 invoices and monetary value 138.90. Potential Customers: average recency 2.50 days, frequency 1.00 invoice and monetary value 37.80. VIP Customers: average recency 3.33 days, frequency 2.00 invoices and monetary value 203.00. Monetary values are expressed in the dataset's price units; the currency should not be claimed unless confirmed from the data source.

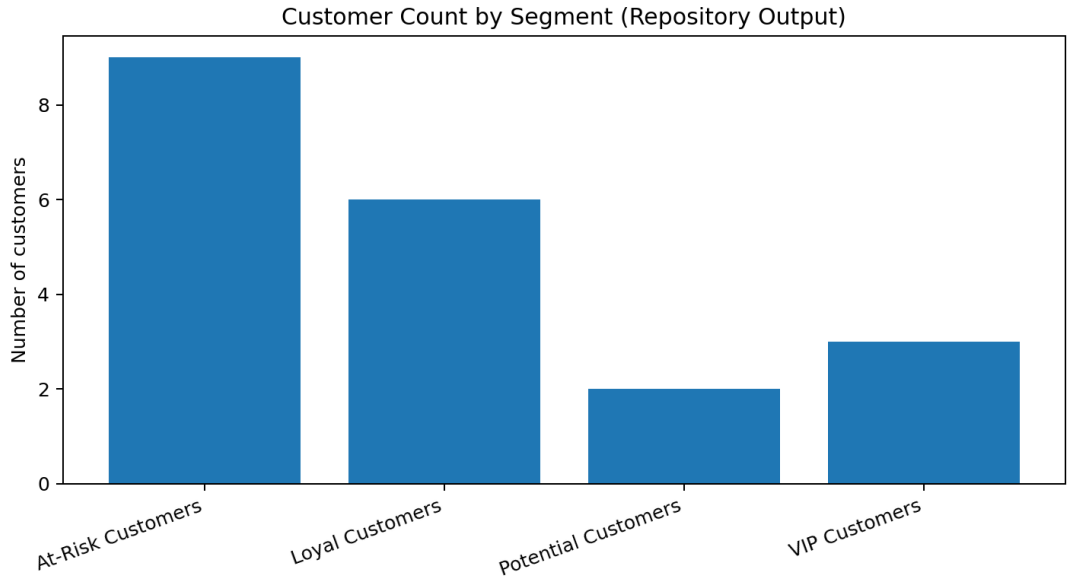
6.4 Interpretation

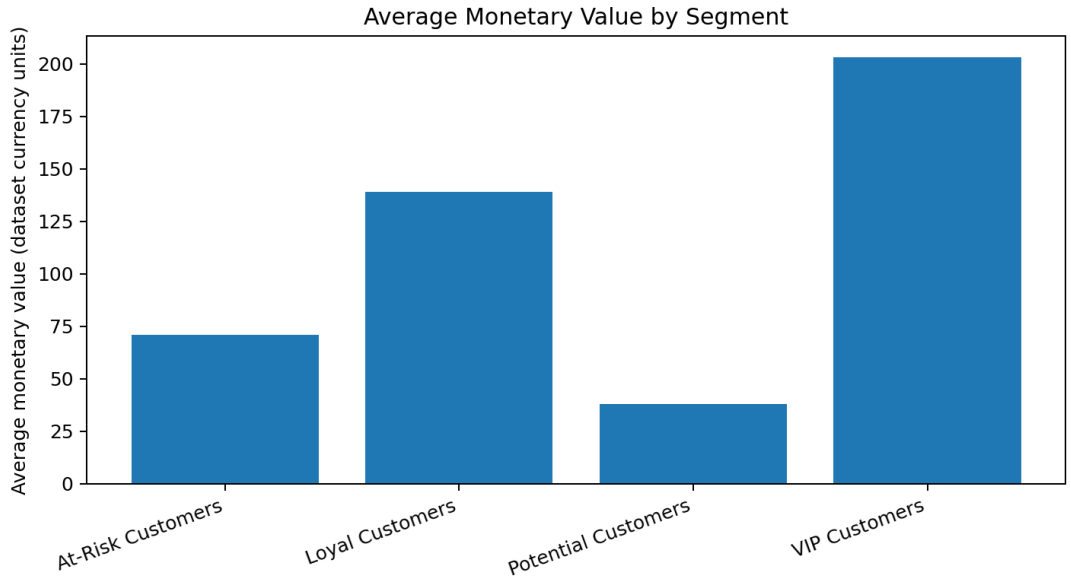
The VIP group has the highest reported average monetary value and relatively recent activity, making exclusive rewards or early access a reasonable test strategy. Loyal Customers show the highest average frequency in the supplied summary, suggesting loyalty points and personalised product recommendations. Potential Customers have low average monetary value and only one invoice on average, so low-cost cross-sell or introductory offers could be tested. At-Risk Customers have the highest average recency, meaning their last recorded purchase is older relative to the other segments; a win-back campaign may be appropriate.

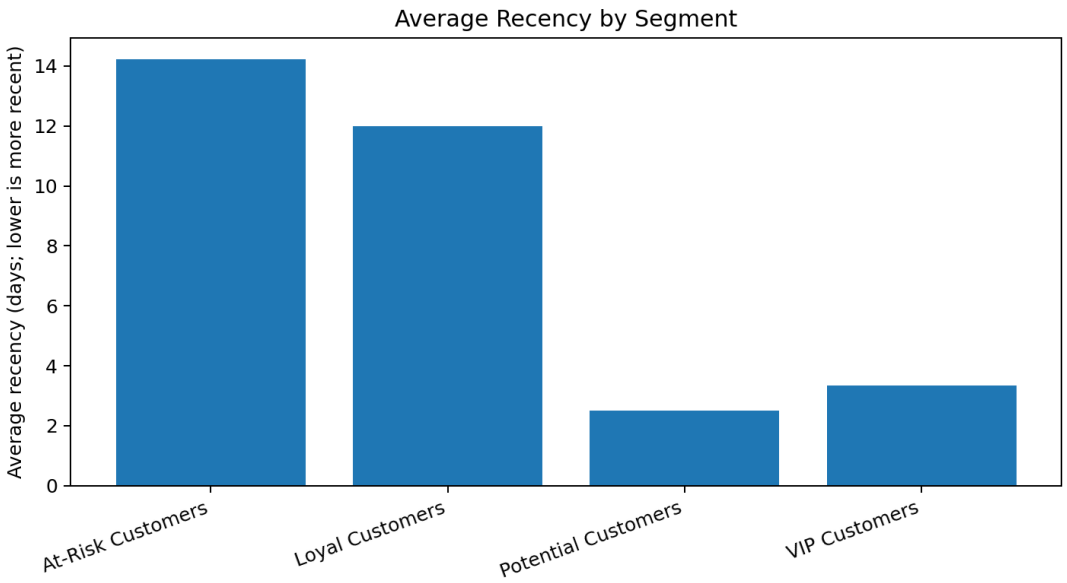
These interpretations are not proof that a campaign will work. They are hypotheses that should be tested using controlled experiments, cost analysis and customer feedback.

6.5 Visual Results

The generated charts in this report reproduce the counts and summary statistics from the committed CSV. The customer-count chart shows the largest segment is At-Risk Customers in the sample. The monetary-value chart shows the highest average spending for VIP Customers. The recency chart shows At-Risk Customers have the largest average recency, while Potential Customers have the smallest. The main program also produces plots for monthly sales, countries, RFM distributions, elbow inertia, cluster scatter plots and segment comparisons.







6.6 Evaluation

The code computes a silhouette score after clustering, but the committed summary CSV does not store this score. Therefore, this report does not invent a numerical value. The score should be copied from a fresh program run and interpreted alongside the elbow curve and business meaning of the clusters. The current K=4 setting is fixed in the code after plotting inertia; the elbow curve alone does not automatically change K.

6.7 Practical Recommendations

Use VIP labels to test premium service or early access. Offer loyalty points to frequent customers. Test product bundles or a next-purchase incentive for potential customers. For at-risk customers, test a respectful re-engagement message and a time-limited offer. Avoid excessive messaging, respect opt-outs and evaluate campaign performance using incremental outcomes rather than only open or click rates.

CHAPTER 7: TESTING AND VALIDATION

7.1 Testing Strategy

Testing should cover file loading, schema validation, data cleaning, RFM calculations, clustering, output writing and edge cases. Tests should be performed on a copy of the data so the original input remains unchanged. The following test cases are suggested for the submitted project.

7.2 Test Cases

Test 1 — valid CSV: expected behaviour is successful loading and column validation. Test 2 — CSV absent but Excel present: the program should load the Excel file. Test 3 — both files absent: the program should raise FileNotFoundError. Test 4 — missing required column: the program should raise ValueError listing the missing field. Test 5 — duplicate transaction: duplicate rows should be removed. Test 6 — missing CustomerID: the row should be excluded. Test 7 — cancellation invoice beginning with C: the row should be excluded. Test 8 — zero or negative quantity or price: the row should be excluded. Test 9 — valid customer transactions: RFM values should match hand-calculated expectations. Test 10 — successful clustering: cluster assignments and output CSV files should be produced.

7.3 Validation Checks

The generated segment counts should add up to the number of customers in the RFM table. Each customer should have one cluster and one segment label. Recency should be non-negative for the chosen analysis date. Frequency should be a positive count of unique invoices. Monetary should equal the sum of valid transaction values under the chosen cleaning policy. Output files should be readable and contain the expected columns.

7.4 Known Risks

The fixed number of clusters may not be optimal for every dataset. The business-labeling heuristic can create misleading labels if clusters are unusual or the cluster summary changes. Removing negative quantities may exclude legitimate returns. The sample is small and may not reflect real customer diversity. There is no automated test suite in the current repository. The charts are displayed interactively through Matplotlib settings, so on some environments the plotting backend may need adjustment.

7.5 Suggested Improvements to Testing

Add unit tests for preprocessing and RFM calculations, integration tests for the complete pipeline, and checks for empty datasets or customers with identical RFM values. Validate results against a small hand-calculated fixture. Record a machine-readable run summary and use a fixed test dataset in continuous integration. For larger data, test memory use and execution time.

CHAPTER 8: CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The E-Commerce Customer Segmentation project demonstrates an end-to-end unsupervised learning workflow using retail transaction data. It performs data validation and cleaning, derives RFM features, applies log transformation and standardisation, clusters customers with K-Means and assigns interpretable business labels. It generates visualisations and exports both customer-level and segment-level summaries. The repository output illustrates four segments across 20 customers, providing a clear demonstration of the workflow.

The project meets its educational objective of applying Python and machine-learning concepts to a practical business problem. However, its small sample size and heuristic segment naming mean that results should be treated as illustrative. A real business deployment would require larger validated data, more rigorous cluster selection, privacy controls, campaign experiments and ongoing monitoring.

8.2 Limitations

The main limitations are the small sample, possible mismatch between sample data and real retail behaviour, fixed K=4, heuristic segment naming, limited automated tests and lack of a user interface or deployed service. The model uses only three behavioural features and does not include product preferences, margins, returns-adjusted revenue, customer lifetime value or channel behaviour. Monetary values should be interpreted only after confirming the currency and data provenance.

8.3 Future Scope

Future work can include using a larger, well-documented retail dataset; explicitly modelling returns; comparing K-Means with hierarchical clustering, DBSCAN or Gaussian mixture models; choosing K through multiple metrics; adding customer lifetime value and product-category features; creating an interactive Streamlit dashboard; scheduling periodic data refreshes; adding a database layer; and generating downloadable campaign lists. A deployment-ready version should include access control, anonymisation, audit logs and clear data-retention rules.

8.4 Ethical and Privacy Considerations

Customer transaction data can be sensitive. Real deployments should minimise the data collected, remove direct identifiers from analysis outputs where possible, restrict access, secure files, define retention periods and comply with applicable privacy laws and organisational policy. Segment labels should not be used to unfairly exclude customers from essential services. Marketing campaigns should be transparent, proportionate and respect customer preferences.

8.5 Final Remarks

This project provides a useful foundation for learning how raw transaction data can be transformed into customer insights. The strongest next step is not merely to increase the number of charts, but to improve data provenance, validate the clustering, document the run environment and test whether segment-specific actions produce measurable business benefits.

REFERENCES

1. scikit-learn Developers, “KMeans,” scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

2. scikit-learn Developers, “StandardScaler,” scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html

3. scikit-learn Developers, “silhouette_score,” scikit-learn documentation. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html

4. pandas Development Team, pandas documentation. https://pandas.pydata.org/docs/

5. NumPy Developers, NumPy documentation. https://numpy.org/doc/

6. Matplotlib Development Team, Matplotlib documentation. https://matplotlib.org/stable/

7. Waskom, M. L., “seaborn: statistical data visualization,” Journal of Open Source Software, 2021.

8. Project repository: parishti71-lgtm, “e-commerce-customer-segmentation-minor.” https://github.com/parishti71-lgtm/e-commerce-customer-segmentation-minor

9. MacQueen, J., “Some Methods for Classification and Analysis of Multivariate Observations,” Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967.

10. Hughes, A. M., Strategic Database Marketing: The Masterplan for Starting and Managing a Profitable, Customer-Based Marketing Program, 4th ed., McGraw-Hill, 2012.

APPENDIX A: REPOSITORY FILES

• E-COMMERCE CUSTOMER SEGMENTATION.py — main analysis pipeline.

• create_dataset.py — helper script that writes a small sample dataset.

• Online Retail.csv and Online Retail.xlsx — transaction inputs.

• requirements.txt — dependency versions.

• customer_segments.csv — customer-level results.

• customer_segment_summary.csv — aggregated segment summary.

• README.md — expand with installation and usage instructions.

Repository URL: https://github.com/parishti71-lgtm/e-commerce-customer-segmentation-minor

APPENDIX B: IMPORTANT CODE LOGIC

Transaction amount

df["TotalAmount"] = df["Quantity"] * df["UnitPrice"]

Recency

analysis_date = df["InvoiceDate"].max() + pd.Timedelta(days=1)

Frequency

Frequency is calculated as the number of unique InvoiceNo values per CustomerID.

Monetary

Monetary is the sum of TotalAmount for each CustomerID.

Feature scaling

Apply np.log1p to each RFM feature, then StandardScaler.fit_transform.

Clustering

KMeans(n_clusters=4, random_state=42, n_init=10)

Evaluation

silhouette_score(rfm_scaled, rfm["Cluster"])

Exports

rfm.to_csv("customer_segments.csv") and final_report.to_csv("customer_segment_summary.csv")

APPENDIX C: VIVA-VOCE QUESTIONS

1. What is customer segmentation?

Grouping customers with similar characteristics or purchasing behaviour.

2. What is RFM?

Recency, Frequency and Monetary value.

3. Why is lower recency often positive?

It means the customer purchased more recently.

4. Why is K-Means unsupervised?

It finds groups without requiring pre-labelled target classes.

5. Why scale RFM features?

To prevent features with larger numeric scales from dominating distance calculations.

6. What does inertia measure?

The sum of squared distances from observations to their assigned cluster centres.

7. What is a silhouette score?

A measure comparing within-cluster similarity with separation from other clusters.

8. Why use log1p?

It reduces skew and the influence of very large positive values.

9. Are VIP and At-Risk direct K-Means outputs?

No. K-Means produces numeric cluster IDs; business labels are added afterward by heuristic rules.

10. What files are generated?

customer_segments.csv and customer_segment_summary.csv.

11. What are limitations of the current project?

Small sample size, fixed K, heuristic labels, and limited automated testing.

12. How can the project be improved?

Use more data, validate K, test multiple algorithms, handle returns carefully and build a dashboard.